Exit Status and Error Codes in Bash (Terminal Style)

Definition:

Every command or script in Bash returns an exit status code that indicates whether it succeeded (0) or failed (non-zero).

Example 1: Checking Exit Status

```
#!/bin/bash
ls /home       # Valid command
echo "Exit status: $?"
ls /no_such_dir  # Invalid command
echo "Exit status: $?"
```

Output:

```
Exit status: 0
Exit status: 2
```

Example 2: Using Exit Code in Condition

```
ping -c 1 google.com > /dev/null 2>&1
if [ $? -eq 0 ]; then
  echo "Internet is working"
else
  echo "No internet connection"
fi
```

Example 3: Short Syntax with && and ||

```
ping -c 1 google.com > /dev/null && echo "Online" || echo "Offline"
```

Example 4: Custom Exit Codes

```
#!/bin/bash
if [ $# -lt 1 ]; then
  echo "Error: No argument provided!"
  exit 1
fi
echo "Argument provided: $1"
exit 0
```

Common Exit Codes

Code	Meaning	Description
0	Success	Command executed fine
1	General error	Catch-all for errors
2	Misuse of shell builtins	Syntax or usage error
126	Command cannot execute	Permission denied
127	Command not found	Invalid command
128	Invalid exit argument	Bad exit number
130	Script terminated (Ctrl+C)	Interrupt signal
255	Exit status out of range	Too high return code

Summary Table

Symbol	Description	Example
$?	Last command exit code	echo $?
exit N	Exit script with code N	exit 1
&&	Run next if success	cmd1 && cmd2
||	Run next if failed	cmd1 || cmd2
set -e	Exit script on any error	-

| return N| Return code from function | return 2 |